

Community Library Membership & Book Lending System

Requirement Understanding:

A Console based application to manage:

- ➔ Members
- ➔ Books
- ➔ Book copies
- ➔ Book categories
- ➔ Members
- ➔ Book borrowings
- ➔ Book borrowings rules management
- ➔ Books returning
- ➔ Late returns fine management
- ➔ Reports and analytics

And the admin can handle:

1. Book Category Management
2. Book Management
3. Member Management
4. Book Borrowings
5. Borrowing Rules Management
6. Fine Management
7. Reports

1. Book Category Management

Before handling books, need book categories, then under book categories can handle the books.

So admin can:

1. Add a book category
2. View all available categories
3. Can make a category as inactive
4. Get categories by category id

2. Book Management

After making categories, admin can handle:

1. Add New Book
2. Add Book Copy
3. View All Books
4. Search By Title
5. Search By Author
6. Search By Category
7. Mark Book Copy As Damaged
8. Mark Book Copy As Unavailable

3. Member Management

1. Adding a new member
2. View all members
3. Search member by phone or email
4. View active members
5. Can activate and deactivate members
6. Can update the membership statuses

Each member should have a membership type such as:

- Basic

- Premium
- Student

→ Before getting this menu, member needs to login into the system.

4. Book Borrowings

Admin can handle:

1. View All Borrowings
2. View All Active Borrowings
3. View All Overdue Borrowings
4. View Borrowings by Member
5. View Member's Active Borrowings
6. Force Return Book (Admin)
7. View Borrowing Statistics

5. Borrowing Rules Management

→ A table itself created for managing borrowing rules management, if future admin can just add new rules and handle the existing rules too.

So admin can:

1. View All Borrowing Rules
2. View Rules by Membership Type
3. Add New Borrowing Rules
4. Update Borrowing Rules
5. Delete Borrowing Rules

6. Fine Management

For late returning books, admin can handle fines as:

1. View All Fines
2. View Unpaid Fines

3. View Fines by Member
4. View Member's Unpaid Fines
5. Generate Fine Report

7. Reports

Admin can generate report for all data.

Admin can get:

1. Dashboard Summary
2. Book Reports
3. Member Reports
4. Borrowing Reports
5. Fine Reports
6. Category Reports

So before coding, the entities and relationships and the database design should be:

Entities:

- ➔ AdminEntity
- ➔ MemberEntity
- ➔ BookCategoryEntity
- ➔ BookCopiesEntity
- ➔ BookEntity
- ➔ BorrowingRulesEntity
- ➔ BorrowEntity
- ➔ FineEntity

Enums:

- ➔ BookBorrowStatus (Borrowed, Returned, Overdue)
- ➔ FinePaymentStatus(Paid, Pending)
- ➔ MembershipStatus(Active, Inactive)
- ➔ MembershipType(Basic, Premium, Student)

→ Then I should create a console based application with DataAccess layer, Business layer, Presentation layer.

→ In **DataAccess** layer, I need to make database configuration then connect with database in local, and create entities and enums, context to make the database design through EntityFramework Core.

→ Then I need to use .env file to store db configuration credentials, so when committing in GitHub repository, I need to ignore the .env file.

→ By using entities, then need to write repository interfaces and their implementations to communicate with database.

→ In **Business layer**, need to write services interfaces and their implementation with validation and verification logics before sending the data to DataAccess layer for database.

→ In Business layer, need to write exception classes also to reuse in services wherever I need.

→ In **Presentation layer**, instead of dumping all code in a single **program.cs** file, need to segregate the things, as like:

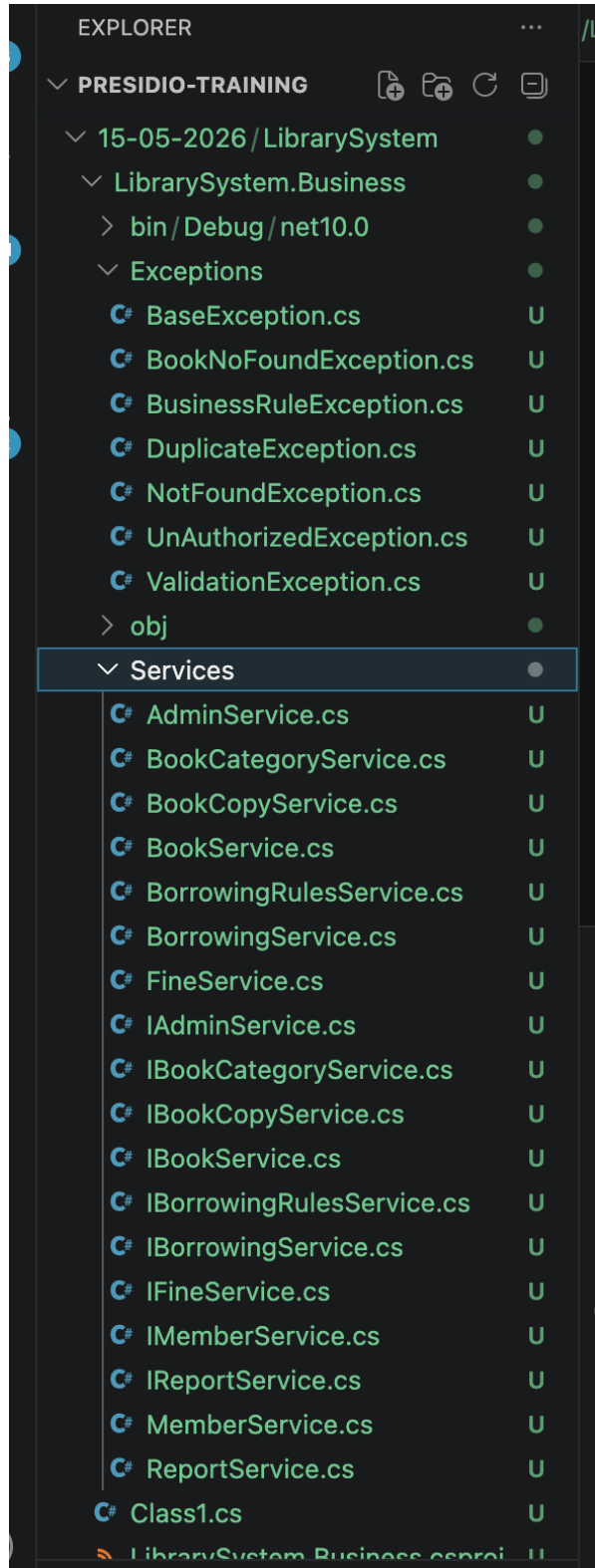
DependencyInjection to manage services,

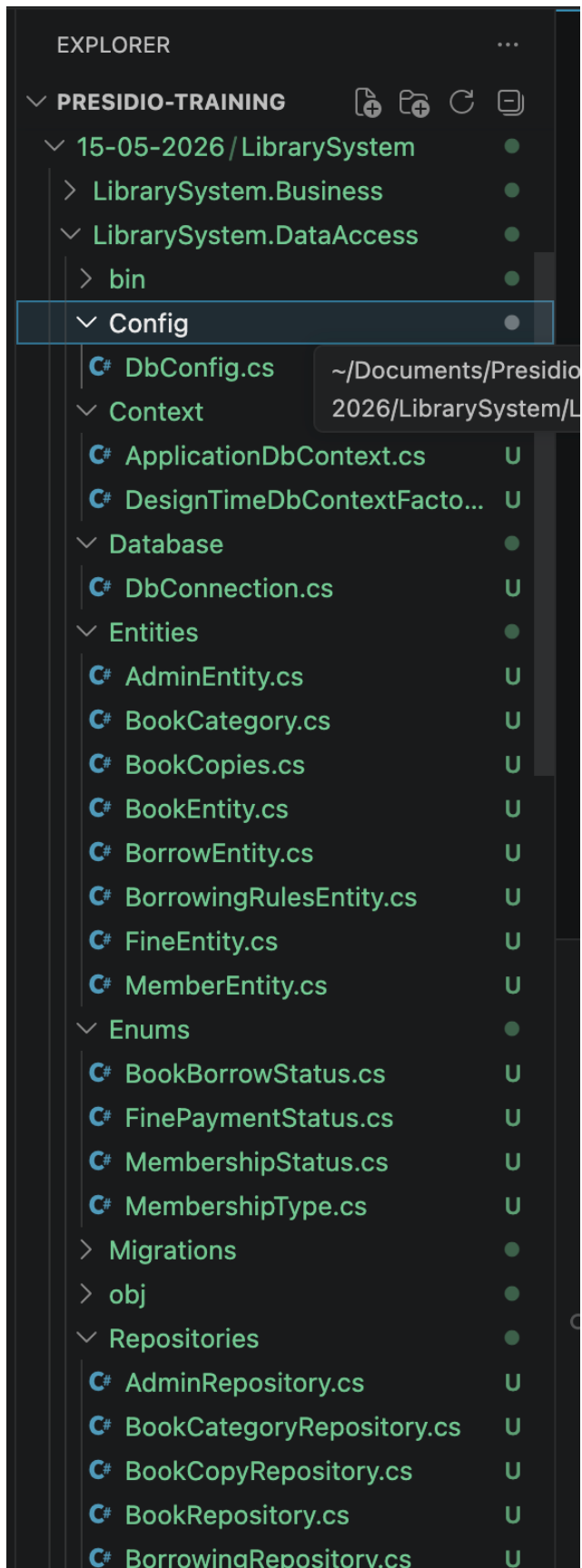
And **Menus**: MemberMenu, BooksMenu, BookBorrowingMenu, etc. → with a menu with implemented functions to get user data and send to services for validation and to database.

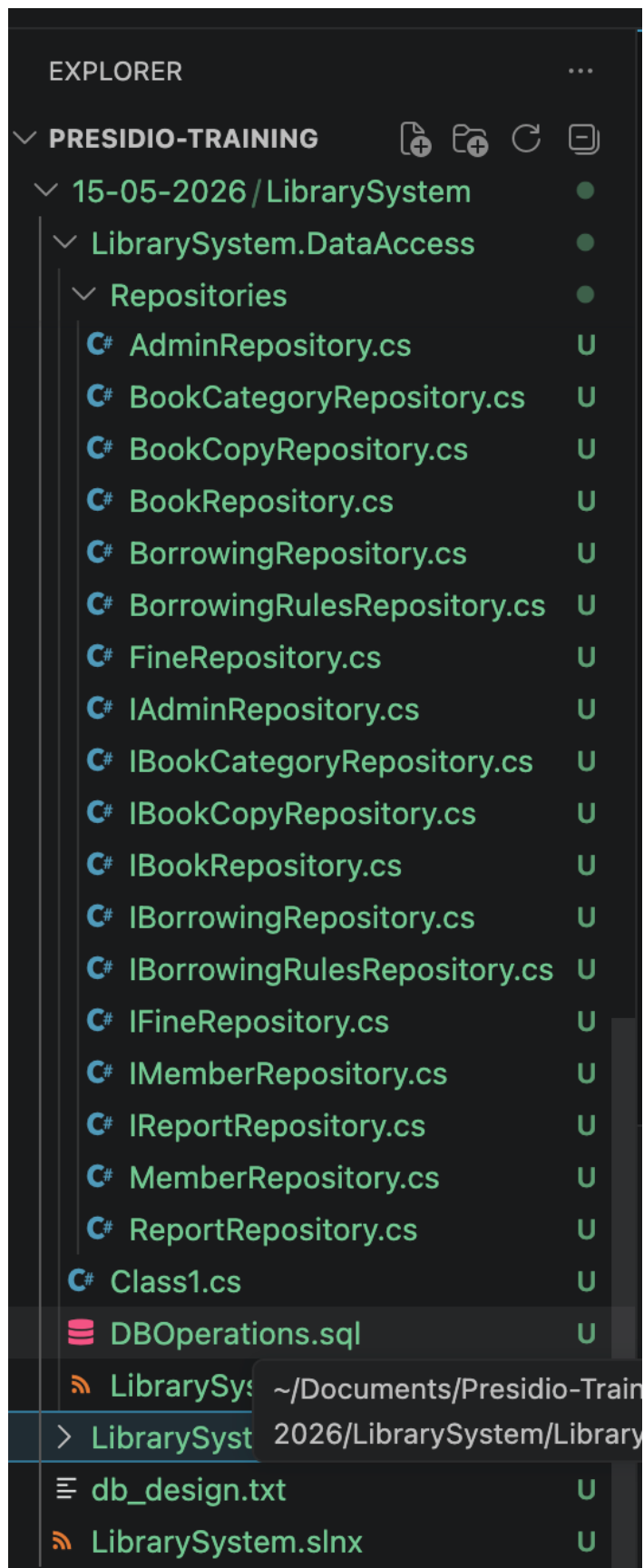
Screens: AdminLogin, MemberLogin, MemberRegistration → to separately make the login and registration process.

Program.cs file to accumulate all the above things and initialize the things to ready to run the app.

The folder structure of the Library management system is:







EXPLORER

...

▼ PRESIDIO-TRAINING

</